

CompassLabs

Full System Design & Architecture Document

A SaaS platform that lets job seekers research, analyze, and compare companies using aggregated data and LLM-powered insights.

Document Version: 3.0

Last Updated: Post Architecture Review — Round 3

GitHub: <https://github.com/VSat08/CompassLabs>

Stack: React + Vite · FastAPI (Python) · Custom JWT Auth · PostgreSQL (Supabase) · Redis (Upstash) · Groq LLM · Vercel + Render.com

Change Log — v2.0 → v3.0

The following decisions were made and locked during architecture review rounds 2 and 3. All items below are final unless explicitly re-opened in a future session.

Status	Item	Change / Decision
✗ Removed	Clerk (third-party auth)	☑ Custom JWT Auth (built from scratch)
✗ Removed	Next.js API Routes as backend	☑ FastAPI (Python) — CONFIRMED
✗ Removed	Prisma ORM	☑ SQLAlchemy (Python ORM)
☑ Confirmed	PostgreSQL via Supabase	Unchanged
☑ Confirmed	Redis via Upstash	Unchanged — dual-DB justified
☑ Confirmed	React + Vite frontend	Unchanged
☑ Confirmed	Groq → OpenAI migration path	Unchanged
☑ Confirmed	FastAPI as backend (final)	Go evaluated & rejected — see §16
📦 Updated	Product name	CompanyIQ → CompassLabs
📦 Updated	GitHub repo	https://github.com/VSat08/CompassLabs
📦 Updated	Deployment	Vercel (frontend) + Render.com (FastAPI)

1. Product Vision & Goals

What is CompassLabs?

CompassLabs is a one-stop SaaS platform where job seekers can search any company — startup, FAANG, listed or unlisted, Indian or global — and get a comprehensive, LLM-synthesized profile including:

- Company history & overview
- Financial performance over a timeline
- Salary benchmarks by role and city
- Employee experience & culture scores
- Pros & cons (LLM-generated from aggregated real data)
- Trending news about the company
- Similar companies recommendation
- Side-by-side comparison of multiple companies

Core Problem Solved

Job seekers currently need 5+ tabs (AmbitionBox, Glassdoor, Crunchbase, Screener, LinkedIn, Google News) to piece together one complete company picture. CompassLabs aggregates all of this into one search, one place, synthesized intelligently by an LLM.

What Makes It 'Out of the Box'

The innovation is NOT the number of companies covered — it is:

1. LLM synthesis layer — raw data from 6 sources becomes one human-readable, honest summary
2. Timeline visualization — a company's arc plotted visually over years
3. Side-by-side comparison — the hero feature no existing tool does well
4. Data flywheel — user-submitted reviews + salary data grows over time, becoming a proprietary moat
5. Everything in one place — the aggregation itself is the product

Design Philosophy

- Apple-inspired: clean, minimal, premium
- Every screen should feel like it belongs in an Apple Keynote
- Data should feel beautiful, not overwhelming
- Mobile-first, but stunning on desktop
- Minimalist typography, generous whitespace, subtle animations

2. User Stories

Guest (Not Logged In)

- Search for a company and see a limited preview (name, founding date, brief overview)
- Prompted to sign up to see the full profile

Free Tier User

- Search any company and see its full profile
- See company history, financial performance, culture scores, pros/cons, and news
- Add up to 2 companies to a comparison list
- See similar companies in a right panel
- Bookmark companies for later
- Submit own salary and review anonymously

Pro Tier User (Future Paid)

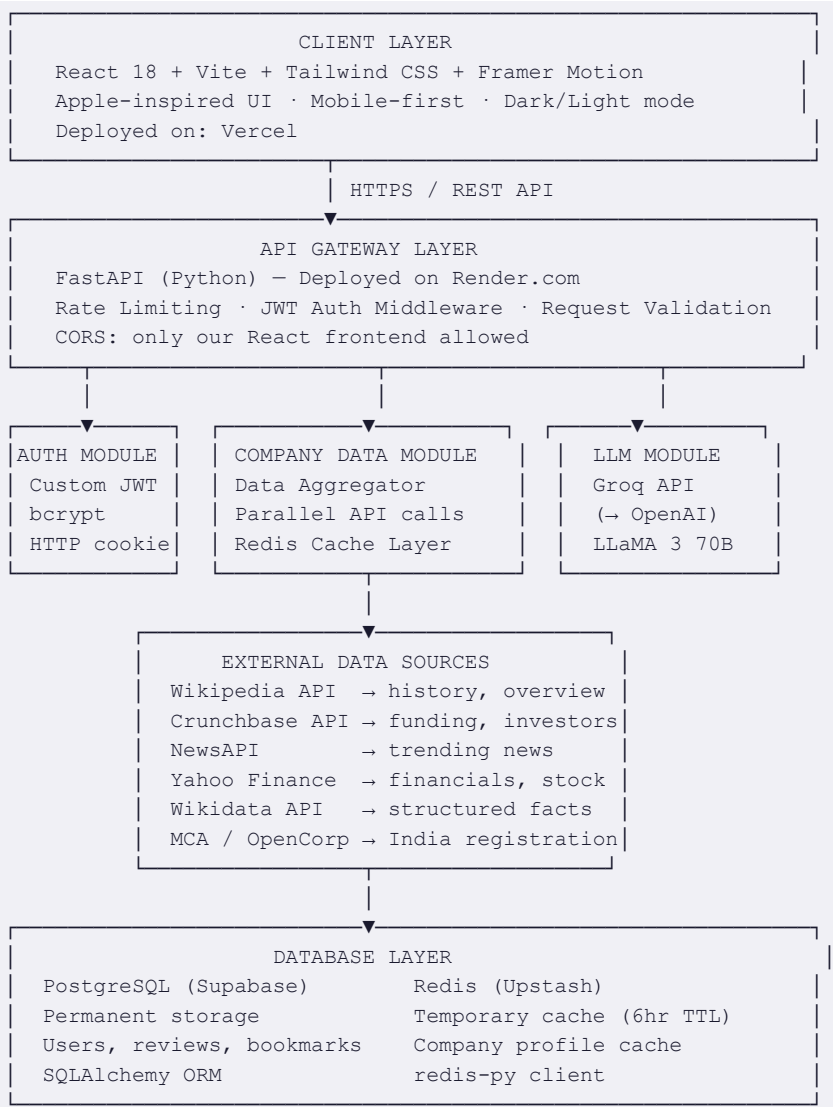
- Compare up to 5 companies side by side
- Export comparison reports as PDF
- Set alerts for company news
- See detailed salary breakdowns by city and experience level

Admin

- View usage analytics
- Manage data source API keys
- Moderate user-submitted reviews

3. System Architecture Overview

Architecture Diagram (Text)



Search Request Flow (Step by Step)

6. User types "Infosys" → presses Enter
7. React sends: GET https://api.compasslabs.app/company/search?q=Infosys
8. FastAPI receives request; JWT Middleware verifies token → extracts user_id
9. Rate limiter checks: has this user exceeded 10 searches/min?
10. Cache check: Redis.get("company:infosys") → HIT: return in <50ms | MISS: proceed
11. Aggregator fires ALL external API calls simultaneously via asyncio.gather()
12. All results merged into one CompanyProfile dict (failures handled gracefully)
13. CompanyProfile sent to Groq LLM with structured prompt
14. Final result = merged data + LLM insights
15. Store in Redis with 6-hour TTL
16. Save/update company record in PostgreSQL (for analytics)

17. Return JSON response to React frontend
18. React renders Apple-style company profile with skeleton loading and animations

4. Tech Stack (All Free / Open Source)

Frontend

Technology	Purpose	Why This?
React 18 + Vite	UI framework	Fast HMR, modern, huge ecosystem
Tailwind CSS v3	Styling	Perfect for Apple-style minimalism
Framer Motion	Animations	Smooth, Apple-like page transitions
TanStack Query	Data fetching & server state	Auto loading/error states, caching
Zustand	Global client state	Lightweight, simple, no boilerplate
React Router v6	Client-side routing	Standard, clean
Recharts	Charts & timeline graphs	Free, composable, React-native
Lucide React	Icons	Clean, minimal, Apple-like icons
React Hot Toast	Notifications	Minimal, elegant toasts
Axios	HTTP client	Clean API calls with interceptors

Backend

Technology	Purpose	Why This?
FastAPI (Python 3.11+)	API framework	Fast, async, auto-docs, perfect for data-heavy + LLM work
SQLAlchemy 2.0	ORM for PostgreSQL	Pythonic, powerful, industry standard
Alembic	Database migrations	Works with SQLAlchemy, clean migration history
Pydantic v2	Request/response validation	Built into FastAPI, type-safe
python-jose	JWT token handling	Industry standard JWT for Python
bcrypt (passlib)	Password hashing	Secure, one-way password storage
httpx	Async HTTP client	Async external API calls
asyncio	Parallel API calls	Call 6 data sources simultaneously
redis-py	Redis client	Cache read/write
uvicorn	ASGI server	Runs FastAPI in production

Database & Infrastructure

Technology	Purpose	Free Tier
Supabase	PostgreSQL hosting	500MB, 50k rows, free forever
Upstash	Redis hosting	10,000 commands/day free
Vercel	Frontend hosting	Free for hobby/personal projects

Render.com	FastAPI backend hosting	Free tier (750 hrs/month)
------------	-------------------------	---------------------------

LLM Strategy

Phase	Provider	Model / Cost
MVP	Groq API	LLaMA 3 70B — Free tier
Growth	OpenAI	GPT-4o mini — ~\$0.15/1M tokens
Scale	OpenAI	GPT-4o + fine-tune — Custom

Migration note: The LLM layer is fully abstracted in backend/app/services/llm/. Switching from Groq to OpenAI is a single-line change in __init__.py. No other file changes needed.

5. Authentication Strategy — Custom JWT (Built From Scratch)

Decision: Build Our Own Auth

We build our own authentication system instead of using Clerk or any third-party service. Three reasons:

- 1. Full data ownership — all user data stays in our own PostgreSQL database
- 2. No third-party dependency — no vendor lock-in, no pricing risk
- 3. Learning — understanding auth deeply is a core developer skill

Auth Flow

- Sign Up → bcrypt hashes password → save to PostgreSQL → issue JWT access (15 min) + refresh (7 days) tokens → set as HTTP-only cookies
- Sign In → verify password hash → if match, issue new JWT tokens in HTTP-only cookies → 401 if no match
- Protected Route → FastAPI middleware intercepts → decodes + verifies JWT → extracts user_id → attaches to request
- Token Refresh → React detects 401 → calls POST /auth/refresh → FastAPI verifies refresh token → issues new access token → request retried transparently
- Sign Out → HTTP-only cookies cleared server-side → refresh token invalidated in DB

Why HTTP-only Cookies (Not localStorage)

	localStorage	HTTP-only Cookie
XSS Attack	❌ Vulnerable — JS can read it	✅ Safe — JS cannot access it
CSRF Attack	✅ Safe	⚠️ Needs CSRF token (we add this)
Auto-sent with requests	❌ Must add manually	✅ Browser sends automatically
Our choice	❌ Rejected	✅ Selected

6. Data Sources & Aggregation Strategy

Source Map by Data Type

- Company Basics (name, founded, HQ, CEO): Wikipedia API (primary) → Wikidata API → OpenCorporates
- Startup & Funding Data: Crunchbase Basic API (free tier) — funding rounds, investors, founded_on
- Indian Company Registration: MCA (data.gov.in) — CIN, registration date, directors
- Financial Performance (listed only): Yahoo Finance via yfinance — "INFY.NS", "TCS.NS", "WIPRO.NS"
- News & Trending: NewsAPI.org (100 req/day free) + Google News RSS (free, no key)
- Employee Reviews & Salaries: Phase 1 — user-submitted via CompassLabs. Phase 2 — potential data partnerships
- Similar Companies: Generated entirely by LLM based on industry, size, geography, stage

Caching Strategy (Redis)

- Key format: "company:{slug}" (e.g. "company:infosys")
- Company profile TTL: 21,600 seconds (6 hours) — data is stable
- News TTL: 3,600 seconds (1 hour) — news changes faster
- Search results TTL: 1,800 seconds (30 min)
- Rate limit counter TTL: 60 seconds (1 min, rolling)
- Cache HIT → return in <50ms | Cache MISS → fetch all sources → merge → LLM → Redis SET → return

7. LLM Integration Strategy

Provider: Groq (MVP) → OpenAI (Growth)

- Groq free tier available, LLaMA 3 70B capable for summarization, very fast inference
- LLM layer fully abstracted: switching provider = one line change in services/llm/__init__.py
- Temperature: 0.3 (factual, less creative), Max tokens: 1500

LLM Output Schema

```
{
  "summary": "2-3 sentence company overview",
  "growth_trajectory": "growing | stable | declining + brief reason",
  "culture_score": "<integer 1-10>",
  "pros": ["pro1", "pro2", "pro3"],
  "cons": ["con1", "con2", "con3"],
  "salary_insight": "general note on compensation",
  "job_seeker_verdict": "honest one-paragraph assessment",
  "similar_companies": ["Company A", "Company B", "Company C"],
  "data_confidence": "high | medium | low"
}
```

Key Prompt Engineering Rules

- System role: 'You are CompassLabs AI analyst. Be factual, balanced, concise. Never invent data.'
- Respond ONLY with valid JSON. No preamble. No markdown.
- If data is limited, acknowledge it honestly — never fabricate
- Description truncated to 500 chars in prompt to control token usage

8. Database Design

Why Two Databases? (PostgreSQL + Redis)

	PostgreSQL	Redis
Type	Relational database	In-memory key-value store
Storage	Disk (permanent)	RAM (temporary)
Purpose	Store everything forever	Cache hot data for speed
Speed	~5-50ms per query	<1ms per operation
Use in CompassLabs	Users, reviews, bookmarks	Company profiles, search cache

Analogy: PostgreSQL is your filing cabinet (permanent). Redis is your desktop notepad (fast, temporary). You need both.

PostgreSQL Tables

- users — id, email, name, password_hash, plan (free|pro), is_verified, created_at, updated_at
- companies — id, name, slug (unique, indexed), data (JSONB), llm_insights (JSONB), search_count, last_fetched_at
- bookmarks — id, user_id (FK), company_id (FK), created_at [UNIQUE(user_id, company_id)]
- reviews — id, user_id, company_id, role, rating_overall/worklife/growth/management (1-5), pros, cons, salary_min/max, is_anonymous
- compare_sessions — id, user_id, company_ids (ARRAY of UUID), created_at
- search_history — id, user_id (nullable), query, company_id (nullable), created_at
- refresh_tokens — id, user_id, token_hash (unique), expires_at, is_revoked

Redis Key Patterns

company:{slug}	→ full company profile JSON	TTL: 6 hours
news:{slug}	→ news articles for company	TTL: 1 hour
search:{query_hash}	→ search result stubs	TTL: 30 min
ratelimit:{user_id}	→ request count (rate limiter)	TTL: 1 min

9. API Design (FastAPI Backend)

Base URL (prod): <https://api.compasslabs.app>

Base URL (dev): <http://localhost:8000>

Auto docs: <http://localhost:8000/docs> ← FastAPI generates this for free

Standard Response Envelope

```
{ "success": true, "data": {...}, "meta": { "cached": true,
"cache_age_seconds": 3240, "data_confidence": "high" }, "error": null }
```

Auth Routes (/auth)

- POST /auth/signup — Register new user. Body: { name, email, password }. Returns: user object + HTTP-only JWT cookies
- POST /auth/signin — Login. Body: { email, password }. Returns: user object + HTTP-only JWT cookies
- POST /auth/signout — Logout. Clears cookies + revokes refresh token in DB
- POST /auth/refresh — Get new access token using refresh token cookie
- GET /auth/me — Get current authenticated user

Company Routes (/company)

- GET /company/search?q={query} — Search companies. Auth: optional (guests get limited results)
- GET /company/{slug} — Full company profile + LLM insights. Auth: required
- GET /company/{slug}/news — Latest news. Auth: required
- GET /company/{slug}/similar — Similar companies. Auth: required
- GET /company/{slug}/reviews — Paginated user-submitted reviews. Auth: required
- POST /company/cache/bust/{slug} — Admin: force refresh cache. Auth: admin only

Compare Routes (/compare)

- POST /compare — Body: { company_slugs: ['infosys', 'tcs'] }. Returns: ComparisonResult. Auth: required
- GET /compare/history — User's past comparisons. Auth: required

User Routes (/user)

- GET/POST /user/bookmarks — Get all bookmarks / Add bookmark { company_id }
- DELETE /user/bookmarks/{company_id} — Remove bookmark
- GET/DELETE /user/history — Get / Clear search history
- POST /user/reviews — Submit review. Body: { company_id, role, ratings, pros, cons, salary_min, salary_max }

10. Frontend Architecture (React + Vite)

Page Routes

- / → Home: hero search, trending companies, how it works
- /company/{slug} → Company profile: full data + right panel similar companies
- /compare → Side-by-side comparison page
- /search?q={query} → Search results list
- /bookmarks → User's saved companies
- /dashboard → User dashboard (history, account settings)
- /sign-in → Sign in page (custom built)
- /sign-up → Sign up page (custom built)

State Architecture

- Zustand (Global Client State): auth slice { user, isAuthenticated, isLoading }, compare slice { selectedCompanies[] — max 2 free / 5 pro }, theme slice { mode: light|dark }
- TanStack Query (Server State): useCompany(slug), useSearch(query) with 300ms debounce, useBookmarks(), useCompare(slugs), useNews(slug)

Key Component Groups

- layout/ — Navbar (GlobalSearchBar, ThemeToggle, UserMenu), Footer, CompareBar (floating)
- company/ — CompanyHero, OverviewSection, TimelineSection, FinancialsSection, CultureSection, ProsConsSection, SalarySection, NewsSection, SimilarCompanies
- compare/ — CompareHeader, CompareTable, CompareRadarChart
- auth/ — SignInForm, SignUpForm, ProtectedRoute
- ui/ — Button, Input, Card, Badge, CompanyCard, ScoreRing, Skeleton, EmptyState, ErrorState, Modal

11. File & Folder Structure

Monorepo Overview

```
compasslabs/
├── frontend/      ← React + Vite (Vercel)
├── backend/       ← FastAPI Python (Render.com)
├── .gitignore
└── README.md
```

Frontend Structure (key paths)

```
frontend/src/
├── pages/          ← One file per route
├── components/
│   ├── layout/     ← Navbar, Footer, CompareBar
│   ├── company/    ← Profile section components
│   ├── compare/    ← Comparison components
│   ├── auth/       ← SignInForm, SignUpForm, ProtectedRoute
│   └── ui/         ← Atomic design system components
├── hooks/          ← useCompany, useSearch, useCompare, useAuth
├── store/          ← Zustand: authStore, compareStore, themeStore
├── api/            ← Axios: client.ts, auth.api.ts, company.api.ts
└── types/          ← TypeScript: company.ts, user.ts, review.ts, api.ts
```

Backend Structure (key paths)

```
backend/app/
├── main.py         ← FastAPI init, CORS, router registration
├── core/           ← config, database, redis, security
├── models/         ← SQLAlchemy table models
├── schemas/        ← Pydantic request/response shapes
├── routers/        ← auth, company, compare, user
├── services/
│   ├── auth_service.py
│   ├── aggregator.py ← asyncio.gather() fan-out
│   ├── cache_service.py
│   ├── data_sources/ ← wikipedia, crunchbase, news, financials, wikidata
│   └── llm/          ← groq.py, openai.py (future), prompts.py
└── middleware/     ← auth_middleware, rate_limiter
```

12. UI/UX Design System (Apple-Inspired)

Color Tokens

Token	Value
--bg-primary	#FFFFFF
--bg-secondary	#F5F5F7 (Apple signature light gray)
--bg-tertiary	#E8E8ED
--text-primary	#1D1D1F (Apple near-black)
--text-secondary	#6E6E73
--accent	#0071E3 (Apple blue)
--success	#34C759 (Apple green)
--warning	#FF9F0A (Apple orange)
--danger	#FF3B30 (Apple red)
Dark: --bg-primary	#000000 (true macOS dark)
Dark: --bg-secondary	#1C1C1E

Typography & Spacing

- Font stack: -apple-system, BlinkMacSystemFont, SF Pro Display, Segoe UI, sans-serif
- Border radius: 12px (cards), 8px (buttons/inputs), 999px (pills/tags)
- Spacing grid: 8pt base (8, 16, 24, 32, 48, 64, 80, 96)
- Max content width: 1200px centered | Main: 70% | Right panel: 30%

Key UI Patterns

- Cards: 12px radius, --shadow-sm, white bg, subtle border
- Skeleton screens — never spinners for loading states
- Framer Motion animations: 200-300ms ease-out, fade + translateY(8px) → 0
- Search bar: 56px tall, centered, SF Pro, subtle shadow — like apple.com
- Section separation: whitespace only, never dividers or horizontal rules

13. Feature Roadmap

MVP — Month 1-2 (Validate the Core Idea)

- Custom JWT auth (signup, signin, signout, refresh)
- Company search with debounced suggestions
- Company profile page (overview, history, LLM summary, pros/cons, news)
- Similar companies right panel
- Basic bookmarking
- Compare 2 companies side by side
- Dark / light mode toggle
- Mobile responsive (375px+)
- 20 pre-seeded companies for reliable demo (Infosys, TCS, Wipro, Razorpay, Zepto, CRED, Swiggy, Zomato, Freshworks, BYJU'S, Ola, Flipkart, PhonePe, Paytm, Google, Microsoft, Amazon, Meta, Apple, Atlassian)

V2 — Month 3-4 (Add Depth & Data)

- User-submitted reviews & salary data
- Financial timeline charts (Yahoo Finance integration)
- Company milestone timeline visualization
- Culture score aggregated from user reviews
- Google OAuth login (social sign in)
- Compare up to 5 companies (Pro tier gate)
- Company news alerts (email digest)
- PDF export of comparison report

V3 — Month 5-6 (Scale & Monetize)

- Stripe subscription (Pro plan billing)
- Advanced salary analytics by city + experience
- Industry-level leaderboards ("Top IT companies in Pune")
- Browser extension (research company while browsing LinkedIn)
- Recruitment agency B2B API access
- College placement cell partnership plans

14. Deployment Strategy (Free Tier)

Service	Platform	Details
Frontend	Vercel	React + Vite. git push → auto-deploy via GitHub. URL: compasslabs.vercel.app
Backend	Render.com	FastAPI + Uvicorn. Free 750 hrs/month. Note: cold starts after 15min idle. Upgrade to \$7/mo for always-on.
Database	Supabase (PostgreSQL)	Free: 500MB storage, 50k rows. Access via DATABASE_URL + SQLAlchemy.
Cache	Upstash (Redis)	Free: 10,000 commands/day. Access via redis-py.
Domain	Vercel subdomain	Free .vercel.app. Later: buy compasslabs.in (~₹800/year) → CNAME to Vercel.
GitHub	VSat08/CompassLabs	https://github.com/VSat08/CompassLabs

Environment Variables — Backend (.env)

```
APP_ENV=development
SECRET_KEY=<generate with: openssl rand -hex 32>
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
DATABASE_URL=postgresql://user:pass@db.supabase.co:5432/postgres
REDIS_URL=rediss://default:token@host.upstash.io:6379
GROQ_API_KEY=gsk_...
CRUNCHBASE_API_KEY=...
NEWS_API_KEY=...
FRONTEND_URL=http://localhost:5173
```

Environment Variables — Frontend (.env)

```
VITE_API_URL=http://localhost:8000
VITE_APP_NAME=CompassLabs
```

15. Security Considerations

Auth Security

- Passwords hashed with bcrypt (cost factor 12) — never stored plain, never logged
- JWT signed with HS256 + secret key from environment (never hardcoded)
- Access tokens expire in 15 minutes — short window limits damage if stolen
- Refresh tokens stored as hashed values in DB (not raw)
- HTTP-only cookies — JavaScript cannot access tokens at all (blocks XSS)
- secure=True + samesite=lax on cookies — safe cross-origin with credentials
- Signout revokes refresh token in DB (true logout, not just cookie clear)

API Security

- All API keys in .env files — NEVER in code, NEVER in Git
- .env in .gitignore — committed as .env.example with placeholder values only
- CORS configured to allowlist only our frontend URL
- All external API calls are server-side only — keys never exposed to browser

Input Validation & Rate Limiting

- All request bodies validated with Pydantic (FastAPI built-in)
- Company name inputs sanitized before LLM prompt injection
- SQL injection impossible via SQLAlchemy ORM (parameterized queries)
- XSS prevention via React's default escaping + HTTP-only cookies
- Redis-based rate limiting: 10 searches/min per authenticated user, 5/min per IP for guests
- LLM response parsed with try/catch — bad LLM output won't crash the API

16. Architecture Decision Record — Go vs FastAPI

This section formally documents the evaluation of Go as a backend stack and the rationale for choosing FastAPI. This is a permanent decision record — not to be revisited without strong new justification.

Decision: FastAPI (Python) — Final

Dimension	FastAPI (Python)	Go (Fiber/Gin)	Winner
Parallel HTTP fan-out (aggregator)	<code>asyncio.gather()</code> — 3 lines	goroutines + <code>WaitGroup</code> + <code>mutex</code> — 30+ lines	FastAPI
LLM SDK ecosystem	groq, openai, langchain — official SDKs	Raw HTTP only — no official Go SDKs	FastAPI ♦♦♦
JSONB schema handling	Python dict → JSONB frictionless	Struct tags + manual marshaling	FastAPI
Auto API docs (Swagger)	Built-in, zero config	Manual or third-party	FastAPI
JWT / bcrypt auth	<code>python-jose</code> + <code>passlib</code> — excellent	<code>golang-jwt</code> + <code>bcrypt</code> — excellent	Tie
Redis client	<code>redis-py</code> — excellent	<code>go-redis</code> — excellent	Tie
Raw throughput	Good (async)	Better (~2-3×)	Go
Cold start / binary size	Moderate	Tiny binary, fast cold start	Go
Time to working MVP	2-3 weeks	6-10 weeks	FastAPI
Relevant at your scale?	Yes — handles 10k users/day easily	Overkill until 100k+ concurrent	FastAPI wins

Why Go Was Rejected for CompassLabs Specifically

4. The aggregator (core product) is cleaner in Python — `asyncio.gather()` vs goroutines + `WaitGroup` + `mutex` is a 4× code complexity difference for zero user-facing benefit
5. The LLM layer (the differentiator) has no Go equivalent ecosystem — Groq, OpenAI, LangChain all have official Python SDKs. In Go, you write raw HTTP clients.
6. JSONB-heavy schema is frictionless in Python — Python dicts map naturally to JSONB. Go requires struct tags + manual marshaling for every field.
7. Scale ceiling is years away — FastAPI + `asyncio` handles 50k users/day comfortably. Go's throughput advantage matters at 100k+ concurrent. That's a revenue problem, not a Day 1 problem.
8. Time to market — Go MVP would take 6-10 weeks. FastAPI MVP takes 2-3 weeks. The 4-8 weeks difference is the entire idea-validation window.

When Go Would Be the Right Answer

- A CLI tool, URL shortener, webhook relay, or file processing microservice
- A high-concurrency queue worker or infrastructure proxy
- A service that does NOT depend on LLM SDKs or heavy data manipulation
- A project where the team already knows Go and Python is the unfamiliar stack

Developer Notes & Learning Path

Phase 1 — Auth (Foundation)

Learn: password hashing, JWT theory, HTTP cookies, middleware, protected routes. Applies to every web app you will ever build.

Phase 2 — FastAPI Backend

Learn: REST API design, async Python, SQLAlchemy ORM, parallel API calls with asyncio. Python + FastAPI is one of the most in-demand backend stacks right now.

Phase 3 — Data Aggregation

Learn: working with multiple APIs, error handling, data merging, caching strategy. Real-world backend engineering most tutorials skip.

Phase 4 — LLM Integration

Learn: prompt engineering, LLM APIs, structured output parsing. The most valuable technical skill in the industry right now.

Phase 5 — Frontend + Full Integration

Learn: React state management, API integration, protected routes on frontend, UX polish.

10 Architecture Principles — Non-Negotiable

9. Start with data pipeline, not UI. Prove you can fetch and merge data for 10 real companies before touching React.
10. Never call Groq/OpenAI directly from a router. Always go through `services/llm/__init__.py`. This is what makes provider switching a one-line change.
11. Never call external APIs from a router. Routers call services. Services call data sources. Keep layers clean.
12. Use `asyncio.gather()` with `return_exceptions=True`. One failing external API must never block the whole response.
13. Cache everything aggressively. On free API tiers, every cache miss costs quota. Redis is your shield.
14. Seed 20 companies into the DB on first run. Demo always works even if all external APIs are down.
15. The compare feature is the hero. Spend extra design time here. Smooth animations, clear visual hierarchy. This is what users will screenshot and share.
16. Mobile first, always. Design every component at 375px first. Most Indian job seekers are on mobile.
17. The company slug is sacred. Always lowercase, hyphenated (e.g. "tata-consultancy-services"). Never change a slug once set — it breaks bookmarks and shared links.
18. Python for backend, TypeScript for frontend. Strict typing in both. Pydantic backend + TypeScript interfaces frontend. This catches 80% of bugs before runtime.